

Predict and Execute Task

Objective

The objective of this task is to improve your ability to **predict Python output before execution** and then verify your understanding by executing the code.

Mastering these exercises is essential for building a deep understanding of Python's underlying mechanics and edge cases. Practice with these foundational concepts will help you confidently navigate tricky Python MCQ tests, excel in competitive programming screenings, and handle technical interview questions with ease.

Important Rules

1. **Predict before executing.**
2. Do not run the code first and then write the prediction.
3. Do not modify the given code.
4. If the code produces an error, record the error and explain **why the error occurs** in your # comment.
5. Your # comment should demonstrate your **understanding of the concept**, not simply repeat the output.
6. Keep the original question number clearly visible.
7. Complete all **50 questions**.
8. Do not copy explanations.
9. Focus on understanding **why** Python produces the particular output.

How to Submit

1. Predict the output for each question before running the code.
2. Execute the code to verify your predictions.
3. Write a comment explaining **why** the code produces that output for each question.
4. **Upload your completed work as a zipped `.ipynb` file strictly in LMS only.**

Concepts Covered

The task covers the following concepts discussed in class:

- Empty set and dictionary trap
- Tuple creation using commas
- Truthiness and `bool()`
- `True`, `1`, and `1.0` as dictionary keys
- `==` vs `is`
- `+` vs `+=`
- List slicing
- `del` – name vs object
- Shallow copy
- Deep copy
- Tuple and dictionary behavior of `args` and `kwargs`
- `print()` vs `return`

Questions

Q1

```
x = dict()
print(type(x))
```

Q2

```
x = set()
print(len(x))
print(type(x))
```

Q3

```
x = {}
print(bool(x))
```

Q4

```
x = set()
x.add(10)
print(x)
```

Q5

```
x = {}
x.add(10)
print(x)
```

Q6

```
x = (25)
print(type(x))
```

Q7

```
x = (25,)
print(len(x))
```

Q8

```
x = (25,)
print(x[0])
```

Q9

```
x = 25, 30
print(type(x))
```

Q10

```
x = (25, 30, 35)
print(x[-1])
```

Q11

```
print(bool(0))
print(bool(10))
```

Q12

```
print(bool(""))
print(bool(" "))
```

Q13

```
print(bool([]))
print(bool([] + [1]))
```

Q14

```
print(bool(()))
print(bool((1,)))
```

Q15

```
print(bool(None))
print(bool("None"))
```

Q16

```
d = {True: "A", 1: "B"}
print(len(d))
print(d)
```

Q17

```
d = {1: "A", True: "B"}
print(d[True])
```

Q18

```
d = {1.0: "A", True: "B"}
print(d)
```

Q19

```
d = {False: "A", 0: "B"}
print(d[0])
```

Q20

```
a = [10, 20]
b = [10, 20]
```

```
print(a == b)
print(a is b)
```

Q21

```
a = [10, 20]
b = a
print(a == b)
print(a is b)
```

Q22

```
a = "hello"
b = "".join(["he", "llo"])
print(a == b)
print(a is b)
```

Q23

```
a = [1]
b = a[:]
print(a == b)
print(a is b)
```

Q24

```
a = int("500")
b = int("500")
print(a == b)
print(a is b)
```

Q25

```
a = [10, 20]
b = a
b.append(30)
print(a)
```

Q26

```
a = [10, 20]
b = a
a = a + [30]
print(b)
```

Q27

```
a = [10, 20]
b = a
a += [30]
print(b)
```

Q28

```
a = [10, 20, 30, 40]
b = a[1:3]
print(b)
```

Q29

```
a = [10, 20, 30, 40]
b = a[::2]
print(b)
```

Q30

```
a = [10, 20, 30]
b = a[:]
b[0] = 99
print(a)
print(b)
```

Q31

```
a = [1, 2, 3]
b = a
del a
print(b)
```

Q32

```
a = [1, 2, 3]
b = a
del a[0]
print(b)
```

Q33

```
a = [1, 2, 3]
b = a
del a[:]
print(b)
```

Q34

```
a = [1, 2, 3]
del a
print(a)
```

Q35

```
import copy

a = [[1, 2], [3, 4]]
b = copy.copy(a)
b[0][1] = 99
print(a)
```

Q36

```
import copy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)
b[0][1] = 99
print(a)
```

Q37

```
import copy

a = [[10], [20]]
```

```
b = a.copy()
a[1].append(30)
print(b)
```

Q38

```
import copy

a = [[10], [20]]
b = copy.deepcopy(a)
a[1].append(30)
print(b)
```

Q39

```
import copy

a = [[1, 2], [3, 4]]
b = copy.copy(a)
print(a is b)
print(a[0] is b[0])
```

Q40

```
a = [[]] * 3
a[1].append(5)
print(a)
```

Q41

```
a = [[]] * 3
a[2].append(10)
a[0].append(20)
print(a)
```

Q42

```
a = [[1]] * 3
a[1].append(2)
print(a)
```

Q43

```
a = [[]] * 4
print(a[0] is a[3])
```

Q44

```
def test(*args):
    print(args)

test(1, 2)
```

Q45

```
def test(*args):
    print(type(args))

test("a", "b", "c")
```

Q46

```
def test(**kwargs):
    print(kwargs)

test(name="Ram", age=25)
```

Q47

```
def test(**kwargs):
    print(type(kwargs))

test(a=10, b=20)
```

Q48

```
def test():
    print(10)

x = test()
print(x)
```

Q49

```
def test():
    return 10

x = test()
print(x)
```

Q50

```
def test():
    print("Python")
    return 100

x = test()
print(x)
```